

ADVANCED MACHINE LEARNING · WORKSHEET 15

Gradient Descent on Logistic Regression and Multiclass Classification

Binary Gradient · Batch Updates · OVR · Softmax · Multiclass Cross-Entropy

TEACHER COPY — Lecture 15

PART I: FROM BINARY CROSS-ENTROPY TO THE PER-EXAMPLE GRADIENT

SECTION 1 Reopen the Unfinished Question: We Know the Loss, but Not the Parameters

For n observations, Lecture 14 ended with

$$J(\boldsymbol{\beta}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)], \quad \hat{p}_i = \sigma(\boldsymbol{\beta}^T \mathbf{x}_i).$$

The formula scores a candidate parameter vector, but standard logistic regression has no convenient closed-form normal equation that directly isolates the minimising weights.

Gradient Descent uses the local slope:

$$\beta_j^{(t+1)} = \beta_j^{(t)} - \alpha \frac{\partial J}{\partial \beta_j}.$$

◆ TAKEAWAY

The derivative supplies direction and steepness; the learning rate controls how far the parameter moves during one update.

SECTION 2 Build the Dependency Chain for One Training Example

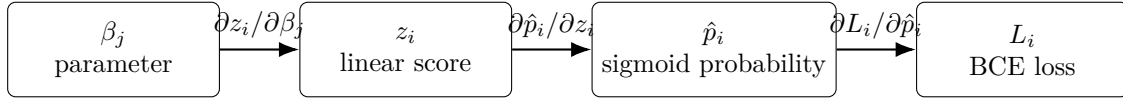
Symbol	Meaning
x_{ij}	Feature j for observation i
β_j	Weight attached to feature j
z_i	Linear score for observation i
$\hat{p}_i, y_i$	Predicted class-1 probability and observed binary label
$L_i, J(\boldsymbol{\beta})$	One-example BCE and average dataset loss

$$z_i = \beta_0 + \sum_{j=1}^d \beta_j x_{ij}, \quad \hat{p}_i = \sigma(z_i), \quad L_i = -[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)].$$

△ WARNING – NOTATION CLEANUP

L_i already includes the leading minus sign, so $J(\beta) = \frac{1}{n} \sum_i L_i$. Do not place a second minus sign outside the sum.

Visual: one weight influences the loss through three local links



$$\frac{\partial L_i}{\partial \beta_j} = \frac{\partial L_i}{\partial \hat{p}_i} \frac{\partial \hat{p}_i}{\partial z_i} \frac{\partial z_i}{\partial \beta_j}.$$

SECTION 3 Differentiate Binary Cross-Entropy With Respect to the Predicted Probability

Holding the observed label y_i fixed and differentiating the two logarithm terms separately gives

$$\frac{\partial L_i}{\partial \hat{p}_i} = -\frac{y_i}{\hat{p}_i} + \frac{1 - y_i}{1 - \hat{p}_i} = \frac{\hat{p}_i - y_i}{\hat{p}_i(1 - \hat{p}_i)}.$$

The second term is positive because $\frac{d}{d\hat{p}_i} \log(1 - \hat{p}_i) = -1/(1 - \hat{p}_i)$, and the BCE expression already has a leading minus sign.

SECTION 4 Differentiate the Sigmoid and the Linear Score

The sigmoid derivative is

$$\hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}} \implies \frac{\partial \hat{p}_i}{\partial z_i} = \hat{p}_i(1 - \hat{p}_i).$$

The factor $\hat{p}_i(1 - \hat{p}_i)$ is largest at $\hat{p}_i = 0.5$ and becomes small near 0 or 1. Probability changes fastest near the decision boundary and slowly in the saturated ends.

A compact derivation is

$$\frac{d}{dz}(1 + e^{-z})^{-1} = -(1 + e^{-z})^{-2}(-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)[1 - \sigma(z)].$$

For the linear score,

$$z_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id} \implies \frac{\partial z_i}{\partial \beta_j} = x_{ij}.$$

For the intercept, define $x_{i0} = 1$, so $\partial z_i / \partial \beta_0 = 1$.

SECTION 5 Let the Chain Rule Collapse to Prediction Minus Target

Multiplying the three local derivatives makes the cancellation visible:

$$\frac{\partial L_i}{\partial \beta_j} = \frac{\hat{p}_i - y_i}{\hat{p}_i(1 - \hat{p}_i)} \cdot \hat{p}_i(1 - \hat{p}_i) \cdot x_{ij} = (\hat{p}_i - y_i)x_{ij}.$$

★ KEY INSIGHT

The gradient is driven by prediction minus target. The feature value determines how strongly that observation influences a particular weight. If x_{ij} is negative, its sign also reverses the direction for that weight.

PRACTICE P1 · DERIVATION, CANCELLATION, AND GRADIENT DIRECTION

(a) For

$$L = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})],$$

differentiate with respect to $\hat{p}$ and combine the two fractions into one.

Expected answer: $\frac{\partial L}{\partial \hat{p}} = -y/\hat{p} + (1 - y)/(1 - \hat{p}) = (\hat{p} - y)/[\hat{p}(1 - \hat{p})]$.

(b) Use $\frac{\partial \hat{p}}{\partial z} = \hat{p}(1 - \hat{p})$ and $\frac{\partial z}{\partial \beta_j} = x_j$ to recover $\partial L / \partial \beta_j$.

Expected answer: The factors $\hat{p}(1 - \hat{p})$ cancel, leaving $\frac{\partial L}{\partial \beta_j} = (\hat{p} - y)x_j$.

(c) Let $y = 1$, $\hat{p} = 0.30$, and $x_j = -2$.

(i) Compute $\hat{p} - y = \mathbf{-0.70}$.

(ii) Compute the gradient contribution $(\hat{p} - y)x_j = \mathbf{+1.40}$.

(iii) Under Gradient Descent, does β_j rise or fall? **It falls.**

Explain why this does not contradict the need to raise the class-1 score.

Expected answer: Because x_j is negative, decreasing β_j increases the product $\beta_j x_j$, which raises z and therefore raises $\hat{p}$.

(d) Compare two positive examples with the same $x_j = 4$: one has $\hat{p} = 0.95$ and the other $\hat{p} = 0.20$. Compute both gradient contributions and explain which observation has more influence.

Expected answer: The contributions are $(0.95 - 1)4 = -0.20$ and $(0.20 - 1)4 = -3.20$. The poorly predicted positive example has the larger magnitude and therefore supplies the stronger upward correction to the score.

PART II: FROM ONE CORRECTION TO BATCH GRADIENT DESCENT

SECTION 6 Move From One Observation to Batch Gradient Descent

Average the per-example losses and their derivatives:

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n L_i, \quad \frac{\partial J}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i) x_{ij}.$$

The scalar Batch Gradient Descent update is

$$\beta_j \leftarrow \beta_j - \alpha \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i) x_{ij}.$$

With a design matrix whose first column can be ones for the intercept,

$$\nabla_{\beta} J = \frac{1}{n} X^T (\hat{\mathbf{p}} - \mathbf{y}), \quad \beta \leftarrow \beta - \alpha \nabla_{\beta} J.$$

Object	Shape	Meaning
X	$n \times (d + 1)$	Design matrix; the first column may contain ones.
β	$(d + 1) \times 1$	Parameter vector.
$\hat{\mathbf{p}} - \mathbf{y}$	$n \times 1$	One prediction error per observation.
$X^T(\hat{\mathbf{p}} - \mathbf{y})$	$(d + 1) \times 1$	One accumulated gradient component per parameter.

One Batch Gradient Descent iteration

1. Choose current parameters and a learning rate α .
2. Compute all scores $\mathbf{z} = X\beta$ and probabilities $\hat{\mathbf{p}} = \sigma(\mathbf{z})$.
3. Compute the average BCE and $X^T(\hat{\mathbf{p}} - \mathbf{y})/n$.
4. Update all parameters simultaneously.
5. Repeat until loss or gradient changes only slightly, or an iteration limit is reached.

SECTION 7 Perform One Complete Numerical Update

Use $x_{i0} = 1$ for the intercept. The two observations are $(x = 1, y = 0)$ and $(x = 2, y = 1)$. Start with $\beta_0 = \beta_1 = 0$, so both scores are zero and both probabilities are 0.5.

i	x_i	y_i	z_i	$\hat{p}_i$	$\hat{p}_i - y_i$
1	1	0	0	0.500	+0.500
2	2	1	0	0.500	-0.500

$$\frac{\partial J}{\partial \beta_0} = \frac{0.5 + (-0.5)}{2} = 0, \quad \frac{\partial J}{\partial \beta_1} = \frac{0.5(1) + (-0.5)(2)}{2} = -0.25.$$

With $\alpha = 0.4$,

$$\beta_0 \leftarrow 0, \quad \beta_1 \leftarrow 0 - 0.4(-0.25) = 0.10.$$

The new scores are 0.10 and 0.20, so the new probabilities are approximately 0.525 and 0.550.

Stage	β_0	β_1	Average BCE
Before update	0.000	0.000	0.693
After one update	0.000	0.100	≈ 0.671

◆ TAKEAWAY

The negative example becomes slightly worse, but the positive example with the larger feature value improves more. Batch Gradient Descent balances all observations, and the average loss falls.

PRACTICE P2 · BATCH GRADIENT AND ONE NUMERICAL STEP

- (a) Recompute the two gradient components above from the error vector $(0.5, -0.5)^T$ and

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}.$$

Expected answer: $X^T(\hat{\mathbf{p}} - \mathbf{y})/2 = [(0.5 - 0.5)/2, (0.5 - 1.0)/2]^T = [0, -0.25]^T$.

- (b) Starting from $\beta = (0.4, -0.1)^T$, suppose the current gradient is $(-0.3, 0.5)^T$ and $\alpha = 0.2$. Compute the simultaneous update.

Expected answer: $\beta_{\text{new}} = (0.4, -0.1)^T - 0.2(-0.3, 0.5)^T = (0.46, -0.20)^T$.

- (c) A student updates β_0 , recomputes the model, and then calculates the derivative for β_1 . Explain the problem.

Expected answer: The components no longer belong to one gradient evaluated at a single parameter vector. Batch Gradient Descent first computes the complete gradient, then updates all parameters together.

- (d) Match each symptom to the most plausible first response.

- (i) Loss oscillates or becomes NaN: **reduce α and use a stable loss implementation.**
- (ii) Loss falls extremely slowly with differently scaled features: **standardise features and tune α carefully.**
- (iii) Loss never changes: **inspect shapes, gradients, and whether parameters are replaced.**

- (e) If the full-data gradient is exactly zero, what does the update do? Does this alone prove that the implementation is correct?

Expected answer: The parameter vector remains unchanged. A zero gradient may indicate a stationary point, but it can also arise from a bug, cancellation, or incorrect data flow, so implementation checks are still required.

△ WARNING – BATCH, STOCHASTIC, AND MINI-BATCH

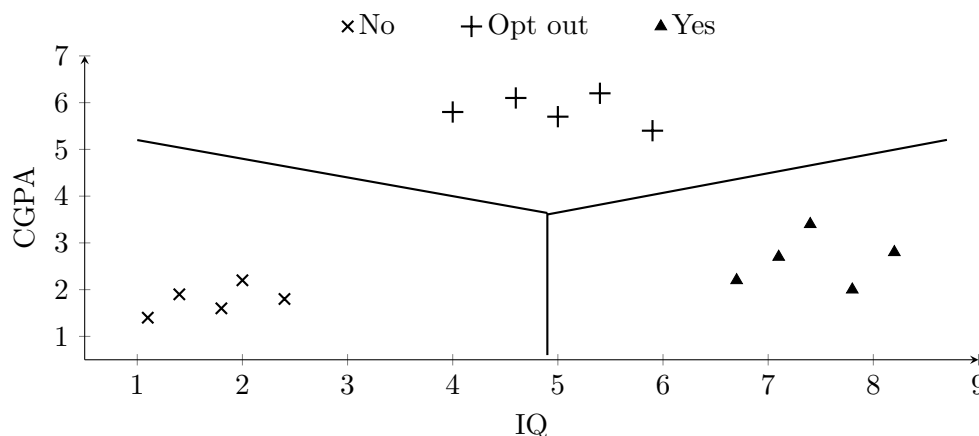
Batch means one update uses all n observations. Stochastic and mini-batch methods alter how many observations contribute to one update; the gradient logic remains the same. They are implementation extensions in this lecture, not separate derivations.

PART III: EXTEND LOGISTIC REGRESSION TO MULTICLASS TARGETS

SECTION 8 Recognise the Binary Limit and Define Multiclass Classification

In this lecture, multiclass classification means that every observation receives exactly one of $K \geq 3$ mutually exclusive labels.

Visual: one input must be assigned to one of three placement classes



Task type	Valid labels for one observation	Example
Binary classification	Exactly one of two classes	Placed or not placed
Multiclass classification	Exactly one of K classes	Yes, No, or Opt out

One sigmoid output naturally gives $\hat{p}$ and $1 - \hat{p}$, so it represents two outcomes. Two principled multiclass routes are:

- **One-vs-Rest (OVR):** fit one binary logistic-regression model for each class.
- **Multinomial or Softmax Regression:** fit all class scores jointly and normalise them into one distribution.

SECTION 9 Construct the One-vs-Rest Training Problems

For three classes, turn the original target into three binary indicator columns:

Original class	Placement = Y	Placement = N	Placement = O
Yes	1	0	0
No	0	1	0
Opt out	0	0	1
Yes	1	0	0

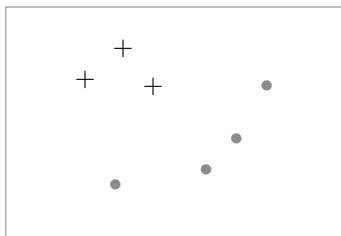
Visual: OVR trains one independent binary model for each class

No vs Rest



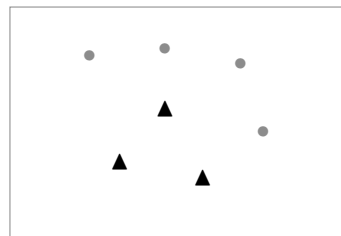
positive: No

Opt out vs Rest



positive: Opt out

Yes vs Rest



positive: Yes

Binary model	Positive label 1	Negative label 0
Model Y	Yes	No or Opt out
Model N	No	Yes or Opt out
Model O	Opt out	Yes or No

Every OVR model has its own parameter vector, score, sigmoid probability, BCE objective, and Gradient Descent updates. For K classes and d features, the conceptual parameter count is $K(d + 1)$.

SECTION 10 Use OVR Models for Prediction—and Interpret Their Outputs Carefully

Suppose the three independently trained models return

$$q_Y = 0.60, \quad q_N = 0.30, \quad q_O = 0.50.$$

q_Y , q_N , and q_O are the sigmoid outputs of the independently trained Yes-vs-Rest, No-vs-Rest, and Opt-out-vs-Rest models, respectively.

They add to 1.40, not 1. For mutually exclusive prediction, choose the largest score or probability:

$$\hat{y} = \arg \max_{k \in \{Y, N, O\}} q_k = Y.$$

Normalizing the probabilities gives

$$\tilde{q}_Y = \frac{0.60}{1.40} = 0.429, \quad \tilde{q}_N = \frac{0.30}{1.40} = 0.214, \quad \tilde{q}_O = \frac{0.50}{1.40} = 0.357.$$

Dividing by the sum preserves the argmax.

OVR strength	OVR limitation
Reuses a reliable binary classifier.	Requires K separately fitted binary models.
Each class receives a dedicated decision problem.	Outputs do not automatically sum to 1.
Allows class-specific weighting or diagnostics.	Models do not directly compete during fitting.

PRACTICE P3 · OVR, TARGET CONSTRUCTION, AND PREDICTION

- (a) The original classes are (Y, O, N, Y, N) . Write the binary target vectors for Models Y, O, and N.

Expected answer: Model Y: $(1, 0, 0, 1, 0)$; Model O: $(0, 1, 0, 0, 0)$; Model N: $(0, 0, 1, 0, 1)$.

- (b) An OVR system returns $(q_Y, q_N, q_O) = (0.42, 0.68, 0.55)$.

- Which class is predicted? **No**.
- Compute the sum: **1.65**.
- Normalise the three values to three decimals: **(0.255, 0.412, 0.333)**.

Why does normalisation not change the predicted class?

Expected answer: All values are divided by the same positive number, so their ordering and argmax remain unchanged.

- (c) With $K = 6$ classes and $d = 4$ input features plus an intercept, how many conceptual OVR parameter entries are fitted?

Expected answer: $K(d + 1) = 6(5) = 30$.

- (d) Decide whether each label set is binary, mutually exclusive multiclass, or multilabel:

- one patient is assigned exactly one of mild/moderate/severe: **multiclass**;
- one image may contain car, person, and bicycle together: **multilabel**;
- an email is spam or not spam: **binary**.

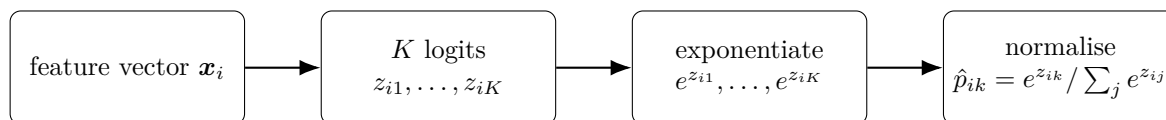
SECTION 11 Build One Joint Multiclass Model With Softmax

For observation i , each class k receives its own linear score, or logit:

$$z_{ik} = \beta_k^T \mathbf{x}_i, \quad k = 1, 2, \dots, K.$$

The logit can be any real number; its meaning is relative to the other class logits.

Visual: softmax exponentiates and normalises all logits together



$$\hat{p}_{ik} = \frac{e^{z_{ik}}}{\sum_{j=1}^K e^{z_{ij}}}, \quad \sum_{k=1}^K \hat{p}_{ik} = 1.$$

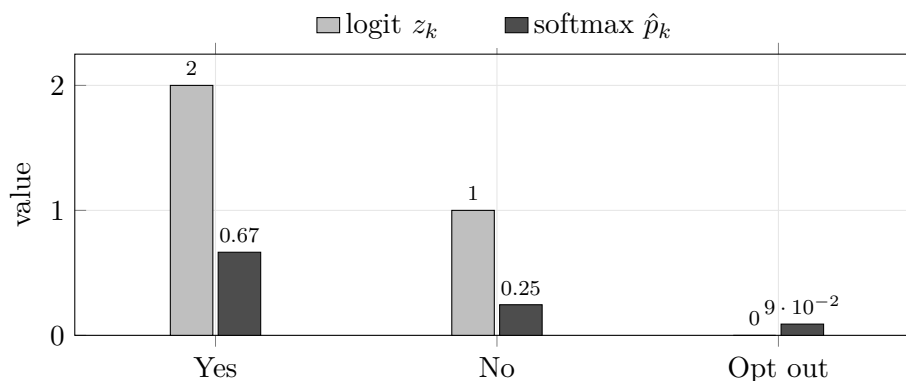
Every probability is positive, prediction uses the largest probability, and adding the same constant to all logits leaves the distribution unchanged. With d features plus an intercept and K classes, the teaching representation contains $(d+1)K$ parameter entries.

SECTION 12 Compute Softmax Probabilities Step by Step

For logits $(2, 1, 0)$,

Class	Logit z_k	e^{z_k}	Softmax probability
Yes	2.0	7.389	$7.389/11.107 = 0.665$
No	1.0	2.718	$2.718/11.107 = 0.245$
Opt out	0.0	1.000	$1.000/11.107 = 0.090$

Visual: relative scores become one probability distribution



SECTION 13 Derive Multiclass Cross-Entropy From One-Hot Labels

Let $y_{ik} = 1$ for the class that occurred and 0 for every other class. The per-example loss is

$$L_i = - \sum_{k=1}^K y_{ik} \log(\hat{p}_{ik}) = - \log(\hat{p}_{i,\text{true}}).$$

Observed class	One-hot target [Y, N, O]	Loss that survives
Yes	[1, 0, 0]	$-\log(\hat{p}_Y)$
No	[0, 1, 0]	$-\log(\hat{p}_N)$
Opt out	[0, 0, 1]	$-\log(\hat{p}_O)$

For the earlier distribution (0.665, 0.245, 0.090), true class No gives $-\log(0.245) \approx 1.407$, whereas true class Yes gives $-\log(0.665) \approx 0.408$. The dataset objective is

$$J(B) = \frac{1}{n} \sum_{i=1}^n L_i = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\hat{p}_{ik}).$$

This is the negative average log-likelihood of a categorical target distribution, paralleling the Bernoulli interpretation of BCE.

SECTION 14 Choose Between OVR and Multinomial Logistic Regression

Dimension	One-vs-Rest	Multinomial / Softmax
Training structure	K independent binary models	One joint K -class objective
Outputs	K independent sigmoid values	K coupled probabilities
Sum to 1?	Not automatically	Yes, by construction
Mutually exclusive prediction	Largest class score/probability	Largest softmax probability
Class interaction	No direct competition during fitting	All classes share one denominator
Interpretation	Class k versus rest	One coherent distribution over classes

Use OVR when a simple binary reduction, a binary-only estimator, or class-specific weighting and diagnostics are important.

Use Softmax when classes are mutually exclusive and a jointly learned probability distribution is required.

⚠ WARNING – CLASS IMBALANCE

Neither OVR nor Softmax automatically solves class imbalance. Class weights, sampling, metrics, and threshold choices still require attention.

PART IV: ASSUMPTIONS, FEATURE MAPS, AND ONE TRAINING STORY

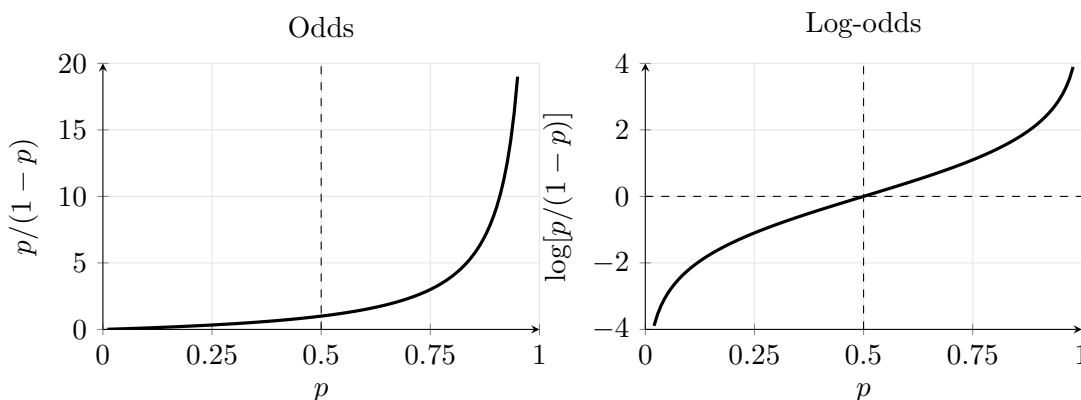
SECTION 15 Logistic-Regression Assumptions: Odds and Log-Odds

The four assumptions carried by the outline are:

1. the target matches the model: binary for binary logistic regression, or one mutually exclusive class for Softmax Regression;
2. observations are independent, or dependence is handled explicitly;
3. continuous predictors have a linear relationship with the log-odds;
4. independent variables do not have severe multicollinearity.

$$\text{odds}(p) = \frac{p}{1-p}, \quad \text{logit}(p) = \log\left(\frac{p}{1-p}\right).$$

Visual: odds and log-odds transform probability in different ways



From the sigmoid,

$$p = \frac{e^{\beta^T x}}{1 + e^{\beta^T x}} \implies 1 - p = \frac{1}{1 + e^{\beta^T x}} \implies \log\left(\frac{p}{1-p}\right) = \beta^T x.$$

Holding other features fixed, increasing x_j by one unit adds β_j to the log-odds and multiplies the odds by e^{β_j} . It does not add a fixed amount to probability.

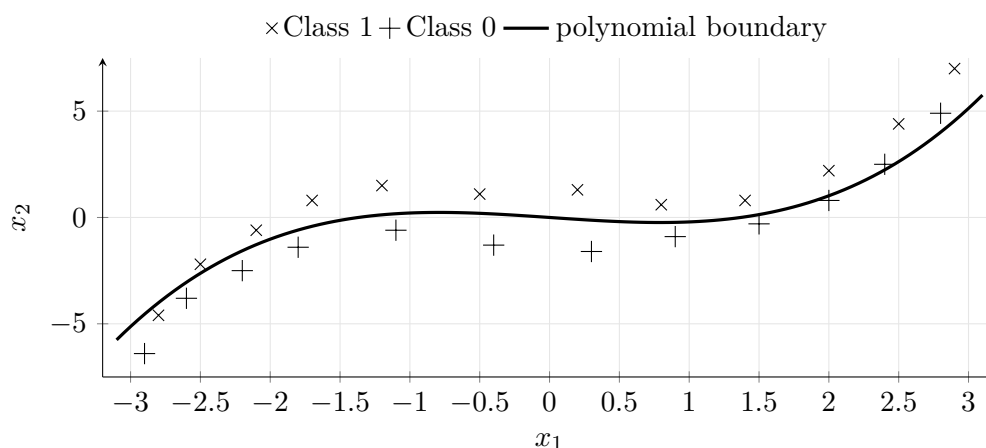
⚠ WARNING – PRACTICAL CHECKS

Grouped observations, repeated measurements, or time-series rows may violate independence. Strongly correlated features can make coefficients unstable. Curvature in log-odds can motivate transformations or polynomial features.

SECTION 16 Create Nonlinear Boundaries With Polynomial Features, Then Control Complexity

With only x_1 and x_2 , the score $\beta_0 + \beta_1 x_1 + \beta_2 x_2$ gives a straight boundary. Adding transformed features keeps the score linear in the parameters while allowing a curved boundary in the original feature plane:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1^2 + \beta_4 x_1 x_2 + \beta_5 x_2^2.$$

Visual: polynomial features can curve the boundary in the original feature space**△ WARNING – THE TASK REMAINS CLASSIFICATION**

Create polynomial features, form a linear score from the transformed features, and pass the score through sigmoid or softmax. Do not describe this as applying polynomial regression first.

Regularization controls the increased flexibility:

$$J_{L2} = J_{\text{data}} + \lambda \sum_{j=1}^d \beta_j^2, \quad J_{L1} = J_{\text{data}} + \lambda \sum_{j=1}^d |\beta_j|.$$

The intercept β_0 is commonly excluded from the penalty. L2 smoothly shrinks large coefficients and often stabilises correlated or expanded feature sets. L1 can drive coefficients exactly to zero, producing a sparse feature set. Feature scaling matters because penalties act on coefficient magnitude.

PRACTICE P5 · ASSUMPTIONS, ODDS, FEATURE MAPS, AND REGULARIZATION

- (a) A coefficient is $\beta_j = \log 1.5$. Holding other features fixed, what happens to the odds when x_j rises by one unit?

Expected answer: The log-odds increase by $\log 1.5$, so the odds are multiplied by $e^{\log 1.5} = 1.5$. This is a 50% increase in odds, not a fixed probability increase.

- (b) For $p = 0.8$, compute the odds and log-odds.

Expected answer: Odds = $0.8/0.2 = 4$; log-odds = $\log 4 \approx 1.386$.

- (c) Diagnose each modelling concern and connect it to an outline assumption:

- (i) repeated measurements from the same student;
- (ii) two predictors that are almost exact linear combinations;
- (iii) a curved relationship between a predictor and empirical log-odds.

Expected answer: (i) threatens independence; (ii) is severe multicollinearity; (iii) violates linearity in log-odds and may motivate transformations or polynomial features.

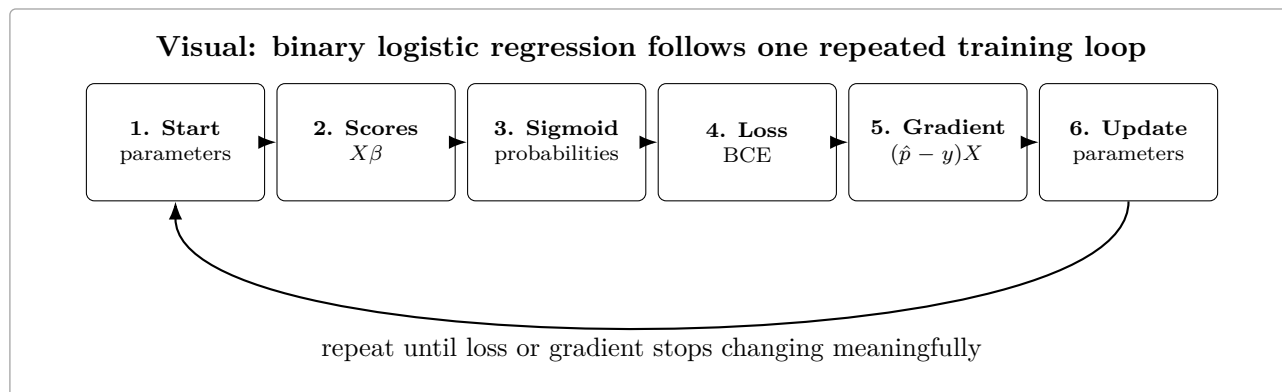
- (d) Explain why a quadratic decision boundary can still come from a model that is linear in its parameters.

Expected answer: After mapping inputs to $(x_1, x_2, x_1^2, x_1x_2, x_2^2)$, the score is a linear combination of the coefficients even though the boundary is curved in the original x_1 - x_2 plane.

- (e) A polynomial expansion creates many weak, possibly irrelevant terms. Compare the mathematical effects of L1 and L2.

Expected answer: L2 smoothly shrinks coefficients and stabilises the expanded set; L1 can set some coefficients exactly to zero and therefore performs sparse feature selection.

SECTION 17 End the Lecture as One Continuous Training Story



1. L14 supplied the probability model and loss; L15 converts that objective into parameter updates.
2. Binary logistic regression produces the correction $(\hat{p} - y)\mathbf{x}$, averaged across the dataset.
3. OVR fits K independent binary models; Softmax fits K class scores jointly.
4. One-hot cross-entropy keeps $-\log$ of the true-class probability.
5. The linearity assumption concerns log-odds, while polynomial features can curve the boundary.
6. Regularization adds a complexity penalty without changing the sigmoid or softmax output rule.

◆ TAKEAWAY

BCE $\rightarrow$ Chain Rule $\rightarrow (\hat{p} - y)\mathbf{x} \rightarrow$ Batch update $\rightarrow$ OVR or Softmax $\rightarrow$ multiclass cross-entropy $\rightarrow$ assumptions, feature maps, and regularization.

Misconception	Correction
"OVR probabilities must sum to 1."	They are independent outputs; softmax enforces a sum of 1.
"Probability must be linear in $\mathbf{x}$."	Log-odds are linear in the predictors.
"Softmax automatically makes curved boundaries."	Linear logits give linear boundaries in the supplied feature space.
"Update one weight before computing the next derivative."	Compute one complete gradient, then update all weights together.

BY THE END OF THIS SHEET, YOU CAN

- ☐ Derive $(\hat{p}_i - y_i)x_{ij}$ from BCE, sigmoid, and the linear score.
- ☐ Write and execute scalar and vector Batch Gradient Descent updates.
- ☐ Explain simultaneous updates and diagnose common learning-rate symptoms.
- ☐ Distinguish binary, mutually exclusive multiclass, and multilabel targets.
- ☐ Construct OVR binary targets and interpret OVR predictions carefully.
- ☐ Compute softmax probabilities from logits.
- ☐ Reduce one-hot cross-entropy to negative log true-class probability.
- ☐ Choose between OVR and Softmax with technical precision.
- ☐ Connect assumptions, polynomial features, and regularization to logistic regression.

ANSWER KEY AND MARKING GUIDE

P1. $\partial L / \partial \hat{p} = (\hat{p} - y) / [\hat{p}(1 - \hat{p})]$; multiplication by the sigmoid and score derivatives leaves $(\hat{p} - y)x_j$. For $y = 1, \hat{p} = 0.30, x_j = -2$, the error is -0.70 , the gradient is $+1.40$, and β_j falls; the negative feature means this raises the score. The two same-feature contributions are -0.20 and -3.20 .

P2. The outlined gradient is $(0, -0.25)^T$. The supplied update gives $(0.46, -0.20)^T$. All gradient components must be evaluated at one parameter vector before simultaneous replacement. Zero gradient produces no update but does not alone prove correct code.

P3. Targets: Y (1, 0, 0, 1, 0), O (0, 1, 0, 0, 0), N (0, 0, 1, 0, 1). Outputs (0.42, 0.68, 0.55) predict No; the sum is 1.65 and normalisation gives approximately (0.255, 0.412, 0.333). OVR fits 30 parameters for $K = 6, d = 4$.

P5. A coefficient $\log 1.5$ multiplies odds by 1.5. At $p = 0.8$, odds are 4 and log-odds are about 1.386. Repeated rows threaten independence; near-linear predictor combinations threaten multicollinearity; curved empirical log-odds threaten linearity. Polynomial feature maps curve boundaries while remaining linear in coefficients. L2 shrinks; L1 can create exact zeros.